

Spark SQL Core 核心架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：SQL Core (v3.3.1)

生成工具：AI Architecture Analyzer

生成日期：2026年03月25日

第1章 模块概述

1.1 模块定位

Spark SQL Core 是 Apache Spark 结构化数据处理的核心引擎模块，位于 `sql/core` 目录下。它构建在 Catalyst 优化器框架之上，提供了从用户 API (SparkSession、Dataset/DataFrame) 到查询解析、分析、优化、物理计划生成和执行的完整查询处理流水线。该模块是 Spark SQL、DataFrame API、Structured Streaming 以及数据源连接器的统一运行时基础。

`sql/core` 模块包含约 564 个源文件 (563 个 Scala + 1 个 Java)，是 Spark SQL 体系中规模最大、职责最重的模块。它承担了将 Catalyst 的逻辑优化能力转化为实际分布式执行能力的关键桥梁角色。

1.2 核心目标

- 统一编程接口：通过 SparkSession 和 Dataset/DataFrame API 为用户提供类型安全和关系型两种编程范式
- 查询优化流水线：实现 SQL Text -> 解析 -> 分析 -> 优化 -> 物理计划 -> 执行 的完整流水线
- 高性能执行引擎：提供全阶段代码生成 (Whole-Stage CodeGen) 和自适应查询执行 (AQE) 两大性能支柱
- 可扩展数据源：支持 V1/V2 双层数据源体系，可对接 Parquet、ORC、JSON、JDBC、Kafka 等多种格式
- 流批一体：通过 MicroBatchExecution/ContinuousExecution 实现结构化流处理引擎

1.3 核心源文件一览

源文件	规模	核心职责
classic/SparkSession.scala	1302行	Spark SQL 主入口，会话管理，Builder 模式
classic/Dataset.scala	2377行	Dataset[T]/DataFrame 完整 API 实现
execution/QueryExecution.scala	815行	查询执行流水线编排 (analyzed->optimized->physical)
execution/SparkPlan.scala	~600行	物理计划基类，定义 execute/doExecute 框架
execution/SparkStrategies.scala	~1400行	逻辑->物理计划转换策略集合
execution/SparkPlanner.scala	119行	物理计划生成器，定义策略优先级链
execution/SparkOptimizer.scala	127行	继承 Catalyst Optimizer 并扩展 Spark 特有优化
execution/SparkSqlParser.scala	~1500行	SQL 解析器，基于 ANTLR4
execution/WholeStageCodegenExec.scala	~900行	全阶段代码生成算子

源文件	规模	核心职责
execution/adaptive/AdaptiveSparkPlanExec.scala	~1000行	自适应查询执行（AQE）根节点
internal/SessionState.scala	209行	会话级组件容器
internal/SharedState.scala	315行	跨会话共享状态
internal/BaseSessionStateBuilder.scala	~500行	Builder 模式构建 SessionState

第2章 核心架构设计

2.1 整体分层架构

Spark SQL Core 采用经典的分层架构设计，从上到下分为用户 API 层、查询编译层、执行引擎层和数据源层四个主要层次。每一层都有清晰的职责边界和抽象接口。

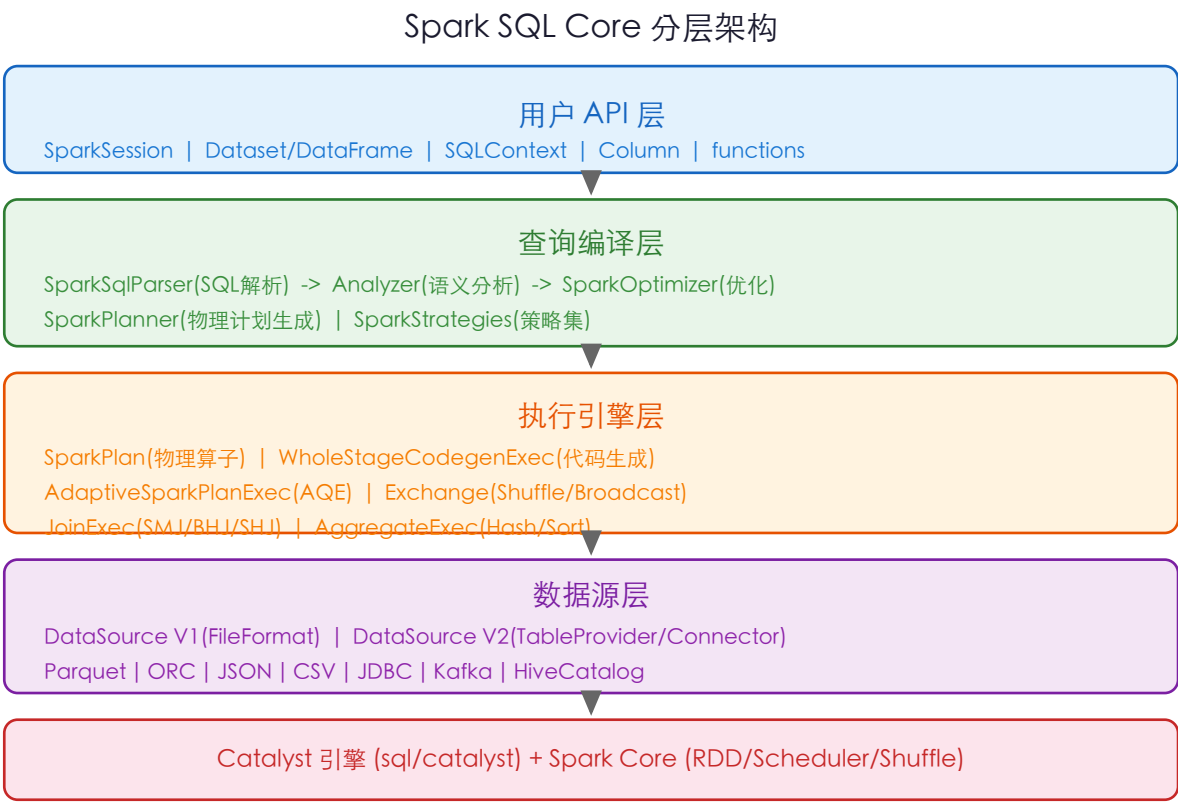


图2-1 Spark SQL Core 整体分层架构图

2.2 会话管理架构

Spark SQL Core 采用 SparkSession -> SessionState -> SharedState
三级会话管理架构。SparkSession 是用户主入口，每个 SparkSession 持有独立的
SessionState（包含 Parser、Analyzer、Optimizer、Planner 等会话级组件），而所有
SparkSession 共享一个 SharedState（包含
CacheManager、ExternalCatalog、GlobalTempViewManager 等全局组件）。

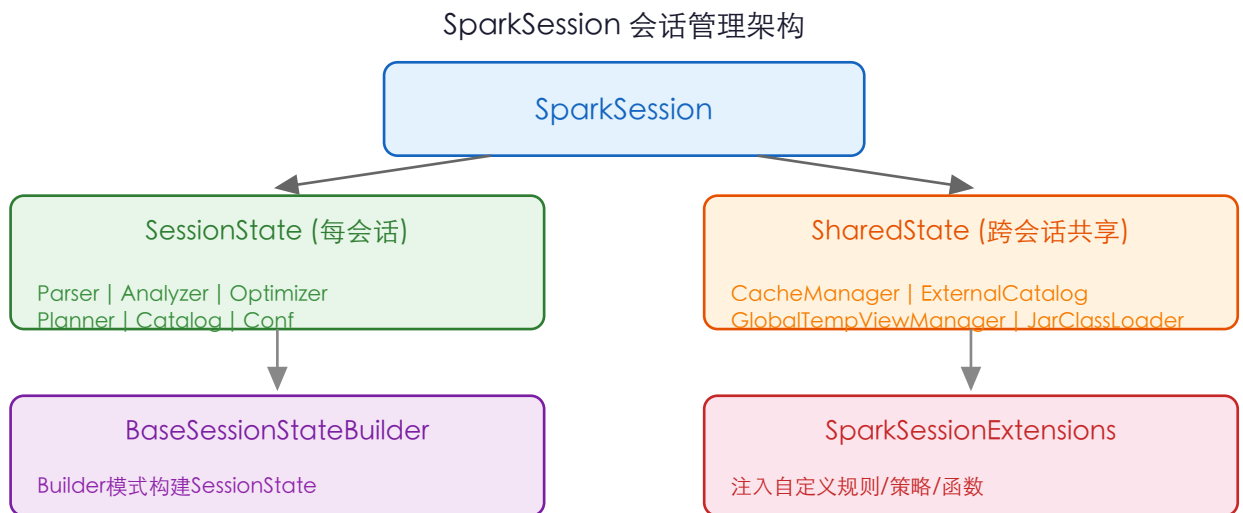


图2-2 SparkSession 会话管理架构图

2.3 查询执行流水线

查询执行流水线是 Spark SQL Core 最核心的处理链路，由 QueryExecution 类编排。每个查询经过六个阶段的 lazy 计算链：logical -> analyzed -> withCachedData -> optimizedPlan -> sparkPlan -> executedPlan。每个阶段都是 lazy val，确保按需计算且仅计算一次。

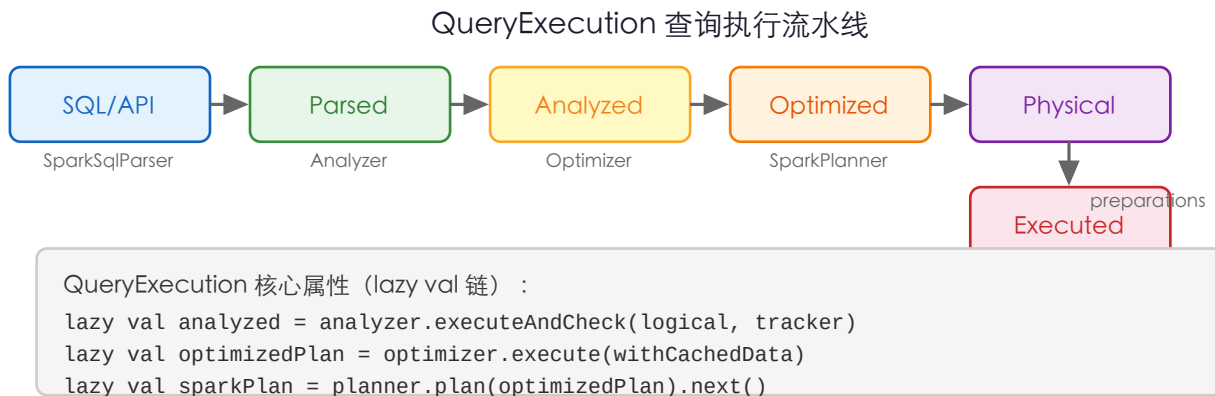


图2-3 QueryExecution 查询执行流水线

2.4 物理计划生成策略链

SparkPlanner 继承自 QueryPlanner[SparkPlan]，通过有序的策略链将逻辑计划转换为物理计划。策略按优先级顺序尝试匹配，第一个能够产生物理计划的策略获胜。这是经典的策略模式（Strategy Pattern）应用。

优先级	策略名称	匹配场景
1	PythonEvals	Python UDF 评估执行
2	DataSourceV2Strategy	V2 数据源读写（Catalog表、文件扫描）
3	FileSourceStrategy	V1 文件数据源扫描（Parquet/ORC等）
4	DataSourceStrategy	V1 通用数据源（JDBC等）
5	SpecialLimits	特殊 Limit 优化（CollectLimit、TakeOrdered）
6	Aggregation	聚合策略（HashAggregate/SortAggregate/ObjectHashAggregate）
7	Window	窗口函数执行策略
8	JoinSelection	Join 算法选择（BHJ/SMJ/SHJ/BNL）
9	InMemoryScans	缓存表扫描
10	SparkScripts	脚本转换
11	BasicOperators	兜底策略（Project/Filter/Sort/Union 等基础算子）

2.5 SparkOptimizer 扩展优化批次

SparkOptimizer 继承自 Catalyst Optimizer，在标准优化规则的基础上增加了 Spark 特有的优化批次。这些优化以 Batch 形式组织，每个 Batch 包含多个优化规则，按序执行。

批次名称	核心规则	功能说明
Optimize Metadata Only Query	OptimizeMetadataOnlyQuery	仅需元数据的查询避免扫描数据
PartitionPruning	PartitionPruning	分区裁剪，减少数据扫描范围
Pushdown Filters (V2)	V2ScanPartitionPruning	V2 数据源过滤下推
Cleanup Filters	CleanupDynamicPruningFilters	清理动态裁剪过滤器
Extract Python UDFs	ExtractPythonUDFFromJoinCondition	从 Join 条件中提取 Python UDF

批次名称	核心规则	功能说明
User Provided Optimizers	ExperimentalMethods.extraOptimizations	用户自定义优化规则注入

第3章 核心类设计模式详解

3.1 SparkSession — Builder 模式 + Facade 模式

SparkSession 是 Spark SQL 的统一入口点，采用 Builder 模式构建实例，同时作为 Facade 统一封装了对底层 SessionState、SharedState 和 SparkContext 的访问。

Builder 模式：SparkSession.Builder 类提供链式调用 API (master/appName/config/enableHiveSupport)，最终通过 getOrCreate() 方法创建或复用 SparkSession 实例。Builder 内部维护 options Map 和 extensions 列表。

Facade 模式：SparkSession 作为统一门面，将 SQL 执行 (sql())、DataFrame 创建 (read/createDataFrame)、Catalog 操作 (catalog)、Streaming (readStream/streams) 等复杂子系统的访问统一暴露给用户。

```
// Builder ■■■■■■  
val spark = SparkSession.builder()  
  .master("local[*]")  
  .appName("MyApp")  
  .config("spark.sql.adaptive.enabled", "true")  
  .enableHiveSupport()  
  .getOrCreate()
```

3.2 QueryExecution — Pipeline 模式 + Lazy Evaluation

QueryExecution 采用 Pipeline 模式编排查询处理的多个阶段，结合 Scala 的 lazy val 实现惰性求值链。每个阶段 (analyzed/optimized/sparkPlan/executedPlan) 都是 lazy val，只在首次访问时触发计算，且缓存结果供后续使用。

关键设计要点：

- lazy val analyzed：调用 Analyzer.executeAndCheck()，进行属性解析、类型推断、函数绑定
- lazy val withCachedData：由 CacheManager 用缓存计划替换匹配的子树
- lazy val optimizedPlan：调用 SparkOptimizer.execute() 应用所有优化规则
- lazy val sparkPlan：调用 SparkPlanner.plan() 使用策略链生成物理计划
- lazy val executedPlan：依次应用 preparations 规则（插入 Exchange、代码生成包装等）

3.3 BaseSessionStateBuilder — Builder 模式 + 模板方法模式

BaseSessionStateBuilder 使用 Builder 模式构建 SessionState，同时通过模板方法模式允许子类（如 HiveSessionStateBuilder）覆写各组件的创建逻辑。每个组件 (analyzer、optimizer、catalog 等) 都是 protected lazy val，子类可重写以提供定制实现。

扩展机制：SparkSessionExtensions 在构建过程中被注入，可以添加自定义的分析规则、优化规则、物理计划策略和函数注册。这实现了开闭原则（对扩展开放，对修改关闭）。

3.4 SparkPlan — 模板方法模式 + 组合模式

SparkPlan 作为所有物理算子的抽象基类，通过模板方法模式定义执行框架。公共方法 `execute()` 负责准备工作（如代码生成上下文设置），然后调用子类实现的 `doExecute()` 进行实际执行。

- `execute()`：模板方法，负责执行前的统一准备（metrics 初始化、RDD 缓存检查）
- `doExecute()`：抽象方法，子类实现具体执行逻辑，返回 `RDD[InternalRow]`
- 组合模式：SparkPlan 通过 `children: Seq[SparkPlan]` 形成算子树，支持递归遍历和转换

3.5 SparkStrategies — 策略模式

SparkStrategies 是策略模式的经典实现。SparkPlanner 持有一组 Strategy 对象，按优先级尝试将逻辑计划节点转换为物理计划节点。每个 Strategy 实现 `apply(plan: LogicalPlan): Seq[SparkPlan]` 方法，返回空序列表示不匹配。

策略选择示例（JoinSelection）：

- 检查是否满足 BroadcastHashJoin 条件（小表 broadcast 阈值）
- 检查是否满足 ShuffledHashJoin 条件（`spark.sql.join.preferSortMergeJoin=false`）
- 检查是否满足 SortMergeJoin 条件（join key 可排序）
- 兜底使用 BroadcastNestedLoopJoin 或 CartesianProduct

第4章 核心流程图

4.1 SQL 查询完整执行流程

从用户提交 SQL 到最终获得结果，经历解析、分析、优化、计划生成、准备和执行六大阶段。下图展示了一条 SQL 查询的完整生命周期。

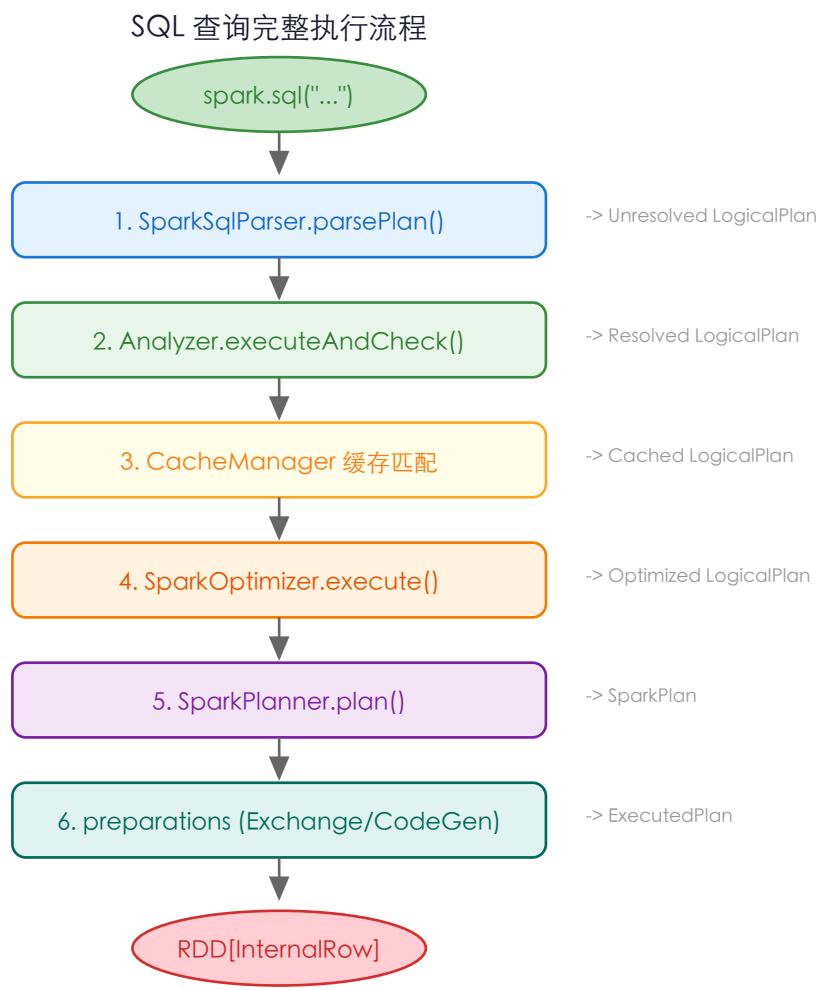


图4-1 SQL 查询完整执行流程

4.2 自适应查询执行 (AQE) 流程

AdaptiveSparkPlanExec 在运行时根据 Shuffle 阶段的实际统计信息（数据量、分区数等）重新优化后续查询阶段。其核心思想是将查询计划按 Exchange 节点切分为多个 QueryStage，每个 QueryStage 执行完毕后收集统计信息，用于重新优化尚未执行的部分。

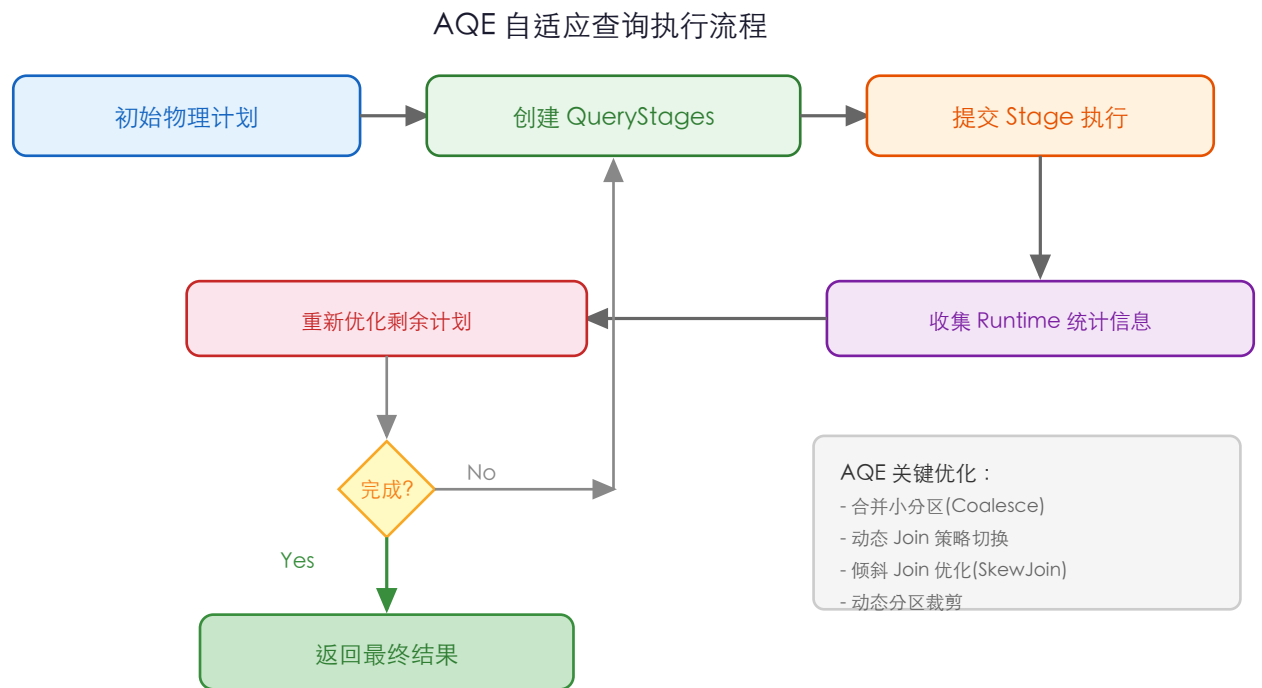


图4-2 AQE 自适应查询执行流程

4.3 全阶段代码生成 (Whole-Stage CodeGen) 流程

Whole-Stage CodeGen 将多个相邻的支持代码生成的物理算子融合为一个 Java 方法，消除了算子间的虚函数调用开销和中间数据的物化开销。核心接口是 CodegenSupport trait，通过 produce()/consume() 的推拉模型生成代码。



图4-3 Whole-Stage CodeGen 代码生成流程

第5章 核心时序图

5.1 spark.sql() 查询执行时序

以下时序图展示了用户调用 `spark.sql()` 后，各组件之间的交互时序。

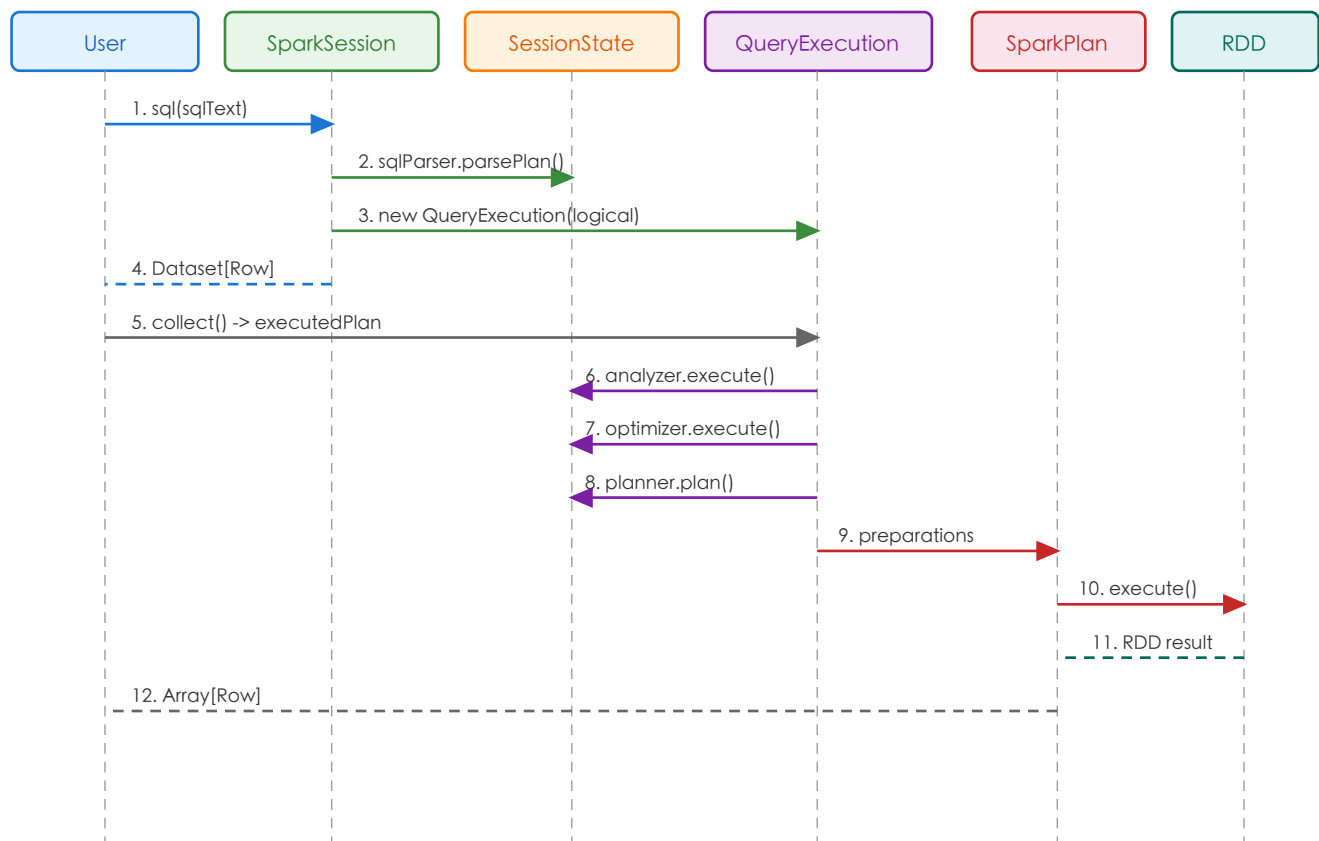


图5-1 spark.sql() 查询执行时序图

5.2 SparkSession 构建时序

SparkSession 的构建涉及 Builder、SparkContext、SharedState 和 SessionState 之间的复杂交互。

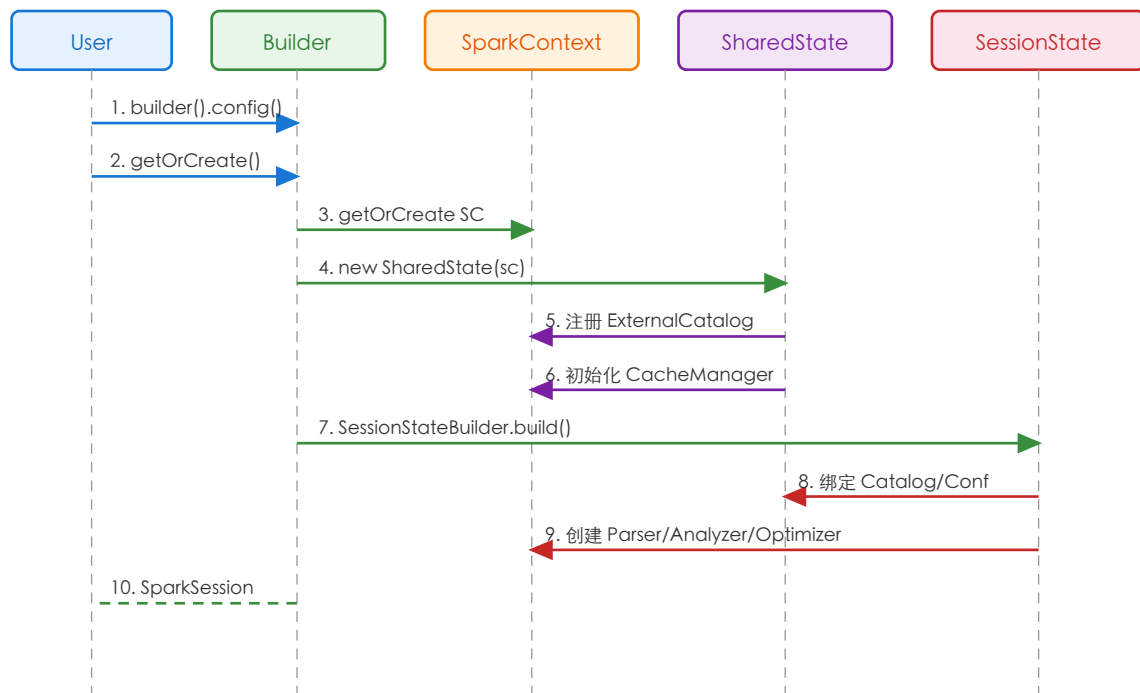


图5-2 SparkSession 构建时序图

第6章 关键场景调用链

6.1 场景一：DataFrame 过滤-聚合-排序查询

触发条件：用户执行 `df.filter(...).groupBy(...).agg(...).orderBy(...).show()`

调用链路：

步骤	类.方法	说明
1	<code>Dataset.filter(condition)</code>	创建 Filter 逻辑算子节点
2	<code>Dataset.groupBy(cols)</code>	创建 <code>RelationalGroupedDataset</code> 包装
3	<code>RelationalGroupedDataset.agg()</code>	创建 Aggregate 逻辑算子节点
4	<code>Dataset.orderBy(cols)</code>	创建 Sort 逻辑算子节点
5	<code>Dataset.show()</code>	触发 Action，调用 <code>QueryExecution</code>
6	<code>QueryExecution.analyzed</code>	<code>Analyzer</code> 解析所有列引用和函数
7	<code>QueryExecution.optimizedPlan</code>	<code>SparkOptimizer</code> 执行优化（谓词下推、常量折叠等）
8	<code>QueryExecution.sparkPlan</code>	<code>SparkPlanner</code> 选择策略（ <code>HashAggregate+Sort+Filter+Scan</code> ）
9	<code>QueryExecution.executedPlan</code>	插入 Exchange（Shuffle）、包装 <code>WholeStageCodegen</code>
10	<code>SparkPlan.execute()</code>	生成 RDD 链并提交到 <code>SparkContext</code> 执行

6.2 场景二：Join 操作的策略选择

触发条件：用户执行 `df1.join(df2, joinExpr, joinType)`

调用链路：

步骤	类.方法	说明
1	<code>Dataset.join(right, joinExpr)</code>	创建 Join 逻辑算子
2	<code>Analyzer.resolveJoin()</code>	解析 Join 条件中的列引用
3	<code>Optimizer.pushdownJoinPredicates()</code>	将过滤条件下推到 Join 子树
4	<code>JoinSelection.apply()</code>	策略选择核心逻辑

步骤	类.方法	说明
5	canBroadcast()?	检查是否有表小于 broadcastThreshold
6	BroadcastHashJoinExec / SortMergeJoinExec	生成对应的物理 Join 算子
7	EnsureRequirements.apply()	检查/插入 Exchange 保证数据分布
8	WholeStageCodegenExec 包装	如果算子支持 CodeGen，融合生成代码

6.3 场景三：数据源读取（V1 FileSource）

触发条件：用户执行 `spark.read.parquet("path")`

调用链路：

步骤	类.方法	说明
1	<code>DataFrameReader.parquet(path)</code>	配置格式为 <code>parquet</code> 并调用 <code>load()</code>
2	<code>DataFrameReader.load()</code>	创建 <code>DataSource</code> 实例
3	<code>DataSource.resolveRelation()</code>	解析数据源，获取 <code>BaseRelation</code>
4	<code>ParquetFileFormat.buildReaderWithPartitionValues()</code>	构建 <code>Parquet</code> 行读取器
5	<code>FileSourceStrategy.apply()</code>	生成 <code>FileSourceScanExec</code> 物理算子
6	<code>FileSourceScanExec.inputRDD</code>	创建 <code>FileScanRDD</code> ，按分区读取文件
7	<code>PartitionPruning + DataSkipping</code>	分区裁剪和数据跳过优化

6.4 场景四：Structured Streaming 微批执行

触发条件：用户执行 `df.writeStream.format("console").start()`

调用链路：

步骤	类.方法	说明
1	<code>DataStreamWriter.start()</code>	创建 <code>StreamingQuery</code>
2	<code>StreamingQueryManager.startQuery()</code>	注册并启动流查询
3	<code>MicroBatchExecution.runActivatedStream()</code>	启动微批循环
4	<code>MicroBatchExecution.runBatch()</code>	执行单个微批
5	<code>Source.getOffset()</code>	获取新数据偏移量
6	<code>IncrementalExecution.executedPlan</code>	增量计划执行（含状态管理）
7	<code>StatefulOperator.stateStore()</code>	从 <code>RocksDB</code> 状态存储读写状态
8	<code>Sink.addBatch()</code>	将结果写入目标 <code>Sink</code>

第7章 配置与扩展

7.1 核心配置属性

配置项	默认值	说明
spark.sql.adaptive.enabled	true (3.2+)	启用自适应查询执行 (AQE)
spark.sql.adaptive.coalescePartitions.enabled	true	AQE 自动合并小分区
spark.sql.adaptive.skewJoin.enabled	true	AQE 倾斜 Join 优化
spark.sql.codegen.wholeStage	true	启用全阶段代码生成
spark.sql.codegen.fallback	true	CodeGen 失败时回退到解释执行
spark.sql.autoBroadcastJoinThreshold	10MB	BroadcastHashJoin 小表阈值
spark.sql.shuffle.partitions	200	Shuffle 默认分区数
spark.sql.sources.parallelPartitionDiscovery.threshold	32	并行分区发现阈值
spark.sql.optimizer.maxIterations	100	优化器最大迭代次数
spark.sql.join.preferSortMergeJoin	true	优先选择 SortMergeJoin

7.2 扩展机制

Spark

SQL

Core

提供了丰富的扩展点，允许用户在不修改源码的情况下注入自定义逻辑。核心扩展通过 SparkSessionExtensions 接口实现。

7.2.1 SparkSessionExtensions 扩展点

扩展点	方法	用途
分析规则	injectResolutionRule	注入自定义的列解析/类型解析规则
检查规则	injectCheckRule	注入自定义的计划验证规则
优化规则	injectOptimizerRule	注入自定义的逻辑优化规则
计划策略	injectPlannerStrategy	注入自定义的物理计划生成策略
后置优化	injectPostHocResolutionRule	注入分析后的修正规则

扩展点	方法	用途
函数注册	injectFunction	注入自定义的内置函数
表函数	injectTableFunction	注入自定义的表值函数

7.2.2 数据源扩展

通过

DataSource

V2

API (TableProvider、ScanBuilder、WriteBuilder)，用户可以实现自定义数据源。V2

API

提供了更细粒度的下推能力 (Filter/Project/Aggregate/Limit 下推) 和更好的并行读写支持。

第8章 设计亮点与总结

8.1 设计亮点

- Lazy Evaluation Pipeline：QueryExecution 使用 lazy val 链实现查询的惰性求值，只有在 Action 触发时才真正执行分析、优化和物理计划生成，避免了不必要的计算开销。
- 策略模式解耦：SparkPlanner 将逻辑到物理的转换抽象为可插拔的策略链，新增策略只需实现 Strategy trait 并注册到 strategies 序列中，完美体现开闭原则。
- Whole-Stage CodeGen 的推拉模型：通过 produce()/consume() 的双向代码生成模型，将多个算子融合为单一紧凑的 Java 方法，消除了传统 Volcano 模型的逐行虚函数调用开销。
- AQE 运行时自适应：突破了传统静态优化的限制，在运行时根据实际数据统计信息动态调整执行策略，包括分区合并、Join 策略切换和数据倾斜处理。
- Builder 模式的层层递进：从 SparkSession.Builder 到 BaseSessionStateBuilder，层层使用 Builder 模式构建复杂对象，保持了构建过程的灵活性和可扩展性。
- V1/V2 数据源双层兼容：通过 DataSource 类同时支持 V1 (BaseRelation) 和 V2 (TableProvider) 两套数据源 API，确保了向后兼容性和向前演进能力。

8.2 设计限制

- 模块体量庞大：sql/core 模块包含 564 个源文件，部分类（如 Dataset.scala 2377行）体量过大，增加了理解和维护的难度。
- Classic/Connect 并存的过渡复杂性：v3.3.1 版本中 classic 包下的 SparkSession/Dataset 正在从顶层 org.apache.spark.sql 包迁移，增加了代码路径的复杂性。
- AQE 与 CodeGen 的协调：AQE 的运行时重优化可能打断 WholeStageCodegen 的融合范围，两个优化机制之间的协调需要精心设计。
- 策略选择的经验性：Join 策略选择主要基于静态阈值（如 broadcastJoinThreshold），缺乏基于代价的精确估算模型。

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★☆	清晰的分层架构和包组织，部分大文件需要进一步拆分
设计模式运用	★★★★★	Builder/Strategy/Template/Pipeline/Facade 等模式运用成熟
可扩展性	★★★★★	SparkSessionExtensions 和 V2 DataSource API 提供丰富扩展点
性能优化	★★★★★	Whole-Stage CodeGen + AQE 代表了查询引擎的最先进优化技术
流批一体	★★★★☆	MicroBatch/Continuous 模式统一了流批处理，但 Continuous 模式仍在实验阶段

评估维度	评分	说明
API 友好度	★★★★★	Dataset/DataFrame API 设计优雅，兼具类型安全和易用性
文档完整度	★★★★☆☆	源码内 ScalaDoc 注释较好，但部分内部组件缺少设计文档

8.4 总结

Spark SQL Core 是 Apache Spark 中设计最为精良的模块之一。它在 Catalyst 优化器框架的基础上，构建了一套完整的从用户 API 到分布式执行的查询处理流水线。模块的核心设计哲学是「惰性求值 + 基于规则的渐进式优化」——通过 lazy val 链实现按需计算，通过可扩展的规则系统实现逐层优化，通过策略模式实现灵活的物理计划生成。

全阶段代码生成（Whole-Stage CodeGen）和自适应查询执行（AQE）是该模块的两大性能支柱，前者通过编译时优化消除运行时开销，后者通过运行时统计信息突破静态优化的局限。两者的结合使 Spark SQL 在大规模数据处理场景下保持了卓越的性能表现。

从架构演进角度看，V1/V2 数据源双层兼容设计展示了良好的向后兼容策略，SparkSessionExtensions 机制则为社区生态的扩展提供了稳固的基础。整体而言，Spark SQL Core 模块是分布式查询引擎设计的优秀范例。